PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Jackfruit Phase-1

Date: 6th September 2026

Software Requirements Specifications for Railway Reservation System

SUBMITTED BY

S.No	Name	SRN
1	PAVANKUMAR H	PES2UG24CS345
2	POOJA KOPPAD	PES2UG24CS352
3	POOJA KANAKAPPANAVARA	PES2UG24CS351
4	POTHINENI DINESH	PES2UG24CS354

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Table of Contents

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Revision History

Name	Date	Reason For Changes	Version
Initial	06-Sep-2026	Initial draft of SRS	1.0

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the functional and non-functional requirements for the Railway Reservation System, a console-based ticket reservation application developed using C/C++. The product is being developed as part of the Mini-Project for the Software Engineering course, 5th Semester, PES University.

This document specifies the complete scope of the system, including core reservation mechanics (search, booking, payment, cancellation), extended features (waitlist/RAC handling, PNR enquiry, booking history, and administrative management), and all constraints applicable to the implementation. It serves as the primary agreement between the development team and the course evaluators on what the final system must accomplish.

1.2 Intended Audience and Reading Suggestions

This document is intended for the following readers:

- Course Instructors / Evaluators: Read all sections to assess requirement completeness, traceability, and alignment with the Software Engineering syllabus.
- Development Team (Team 12): Use Sections 2–5 to understand the full feature set and individual ownership areas. The RTM in Appendix C maps each requirement to design and test artefacts.
- Testers: Focus on Section 5 (System Features, functional requirements) and Appendix C (Requirement Traceability Matrix) to derive test cases.
- Future Maintainers: Sections 2.5, 2.6, and 6 cover constraints and quality attributes relevant for future modifications.

Readers primarily interested in the reservation features may go directly to Section 5. Section 6 covers non-functional quality attributes.

1.3 Product Scope

The product is a standalone, single-user Railway Reservation System built as a Windows/Linux desktop/console application using C++17. The system allows a passenger to search trains, check seat availability, book and pay for tickets, cancel bookings and receive refunds, and enquire about PNR status and booking history. Administrators can maintain train schedules, fares and quotas, and generate operational reports.

The extended scope includes the following capabilities:

- Waitlist and RAC (Reservation Against Cancellation) handling with automatic upgrade on cancellation.
- A configurable, time-based cancellation-fee and refund policy.
- Persistent storage of trains, users and bookings in local data files.
- Administrative management of train schedules, fares, and seat quotas, plus occupancy/revenue reporting.

The system does not support multi-user concurrent access across machines, real payment-gateway integration, or network connectivity. It is an academic project intended to demonstrate the full Software Development Life Cycle as outlined in the course guidelines.

1.4 References

#	Title / Document	Source / Location
1	ISO/IEC/IEEE 29148:2018 – Systems and Software Engineering: Requirements Engineering	IEEE Standards
2	C++17 Standard (ISO/IEC 14882:2017)	https://isocpp.org/
3	C++ Standard Template Library (STL) Reference	https://en.cppreference.com/
4	SE Mini-Project Guidelines (Jackfruit), PES University 2026	Course Moodle Portal
5	GitHub Education	https://github.com/education
6	SonarQube Static Analysis Documentation	https://docs.sonarqube.org/

2. Overall Description

2.1 Product Perspective

The Railway Reservation System is a new, self-contained standalone application. It is not part of any larger system and does not depend on any external server or network service. The application is composed of the following major internal subsystems:

- Auth Manager: Handles passenger and admin registration, login, session and lockout logic.
- Train Search: Queries the train-schedule data store and returns matching trains and fares.
- Seat Inventory: Tracks live seat occupancy per train, date and class, including soft locks.
- Booking Manager: Creates bookings, assigns PNRs, and manages Confirmed/Waitlisted/RAC status.
- Payment Module: Calculates fares and processes payment through a simulated gateway.
- Cancellation Manager: Processes cancellations and calculates refunds per policy.
- Report Generator: Produces occupancy and revenue reports for administrators.

The application interacts with the operating system only for console/text I/O and local file I/O. No network interfaces are required.

2.2 Product Functions

The following is a high-level summary of the major functions the product must provide:

- Allow passengers to register and log in securely.
- Allow passengers to search trains by source, destination and date.
- Display real-time seat availability and fares by class.
- Allow booking of one or more seats with passenger details, generating a unique PNR.
- Process payment and confirm, waitlist, or RAC a booking as appropriate.
- Allow cancellation of a ticket and calculate a policy-based refund.
- Allow PNR status enquiry and viewing of booking history.
- Generate a downloadable/printable e-ticket for confirmed bookings.
- Allow administrators to manage train schedules, fares and quotas.
- Allow administrators to generate occupancy and revenue reports.

2.3 User Classes and Characteristics

User Class	Description	Technical Expertise	Primary Interaction
Passenger	General public using the system to search, book, and manage rail tickets.	Low to Moderate	Booking, cancellation, PNR/history
Administrator	Railway staff who maintain schedules, fares and quotas and review reports.	Moderate	Train/fare management, reports
Course Evaluator / Instructor	Faculty who assess the project for correctness, code quality, and SDLC adherence.	High	All features, code review
Developer (Team Member)	Team 12 members who develop, test, and maintain the system.	High	Full system including internals

2.4 Operating Environment

Component	Specification
Operating System	Windows 10/11 (primary) or Ubuntu 20.04 LTS / Debian-based Linux
Programming Language	C++17 (compiled with GCC 9+ / MSVC 2019+ or equivalent)
Build System	CMake 3.16+ or Makefile
Minimum RAM	512 MB
Interface	Text-based console interface (no GUI framework required)
Storage	At least 50 MB free (executable + data files)
CI Environment	Jenkins pipeline on a GitHub-hosted repository

2.5 Design and Implementation Constraints

- Language Constraint: The entire codebase must be implemented in C++ (C++17 standard). No other languages are permitted for core reservation logic.
- Single-User Only: The system must not require or support a network connection; all data is local to the running instance.
- File Storage: Persistence must use plain-text or binary local files. No external databases are permitted.
- Academic Integrity: All code must be original or appropriately cited. LLM-assisted code must be understood and explainable by every team member.
- Code Quality: SonarQube (or equivalent static analysis) must be integrated into the CI pipeline. No critical code smells or security hotspots are permitted in the final submission.
- Version Control: All source code must reside in a GitHub repository with meaningful commit messages and branch-based development with pull-request reviews.
- Modularity: The codebase must be organised into separate, well-defined modules/classes (e.g., AuthManager, BookingManager, SeatInventory, PaymentModule, CancellationManager) to enable independent development and testing.

2.6 Assumptions and Dependencies

Assumptions:

- It is assumed that the executing machine has a working C++17 toolchain and standard console I/O.
- It is assumed that the user has write permissions to the directory where data files are stored.

- It is assumed that all four team members have access to a development machine with GCC/MSVC configured.

Dependencies:

- CMake 3.16+ – required for platform-independent build configuration.
- Jenkins – required for CI/CD pipeline execution, connected to the GitHub repository via webhook.
- SonarQube (or equivalent) – required for static code analysis as part of the CI stage.
- GoogleTest (gtest) – required for unit test execution and coverage reporting.

3. External Interface Requirements

3.1 User Interfaces

The system provides a text-based console interface organised into the following screens/menus:

- Main Menu: Register, Login, Search Trains, Admin Login, Exit.
- Search & Results Screen: Prompts for source, destination, date; lists matching trains with class-wise fares and availability.
- Booking Screen: Seat/class selection, passenger detail entry, and booking confirmation summary.
- Payment Screen: Fare summary and simulated payment entry, followed by a confirmation/receipt display.
- PNR/History Screen: PNR status lookup and booking-history listing with filters.
- Admin Menu: Manage Trains, Manage Fares & Quota, Generate Reports, Logout.

All prompts and outputs shall use clear, unambiguous text. Error conditions (e.g., invalid input, no seats available) shall be displayed as brief, specific on-screen messages.

3.2 Software Interfaces

Interface	Version	Purpose	Interaction Type
C++ Standard Library (STL)	C++17	Data structures (vector, map, string), file I/O (fstream), random number generation.	Language built-in
GoogleTest	1.14+	Unit testing framework for reservation logic classes.	Compiled test binary
CMake	3.16+	Build system generator for cross-platform compilation.	CLI tool
Jenkins	LTS	CI/CD automation: build, test, static analysis pipeline.	REST/Webhook from GitHub

3.3 Communications Interfaces

The Railway Reservation System is a fully offline, standalone application. It does not require, implement, or use any network or communication interfaces. There are no HTTP calls, sockets, or any other inter-process communication mechanisms.

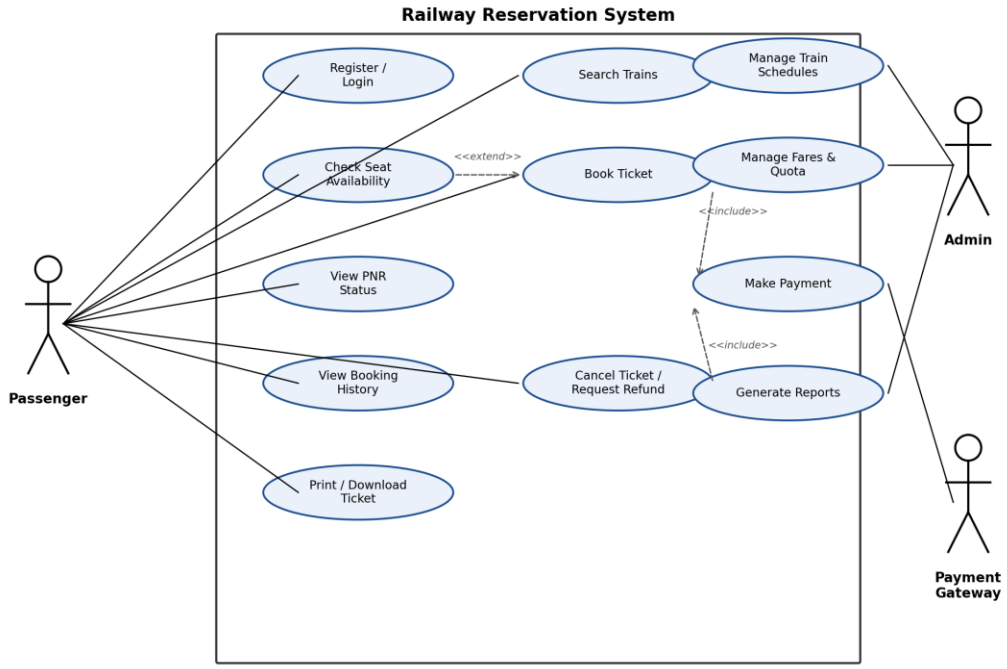
The only external I/O the system performs is local file system access for reading and writing train, user and booking data files (trains.dat, users.dat, bookings.dat) on the host machine, using the standard C++ fstream library.

4. Analysis Models

4.1 Use-Case Diagram

The primary actors are the Passenger and the Admin. The following use cases are supported:

Use Case ID	Use Case Name	Actor	Brief Description
UC-01	Register Account	Passenger	A new passenger creates an account with name, email, phone and password.
UC-02	Login	Passenger	A registered passenger authenticates using email/username and password.
UC-03	Search Trains	Passenger	Passenger searches for trains by source, destination and date.
UC-04	Check Seat Availability	Passenger	Passenger views live seat availability for a chosen train, date and class.
UC-05	Book Ticket	Passenger	Passenger selects seats, enters passenger details and confirms a booking.
UC-06	Make Payment	Passenger	Passenger completes payment through the simulated payment module.
UC-07	Cancel Ticket	Passenger	Passenger cancels a booking via PNR and receives a refund per policy.
UC-08	View PNR Status	Passenger	Passenger enters a PNR to view the current booking status.
UC-09	View Booking History	Passenger	Passenger views past and upcoming bookings.
UC-10	Download/Print E-Ticket	Passenger	Passenger downloads or prints the e-ticket for a confirmed booking.
UC-11	Manage Train Schedules	Admin	Administrator adds, edits or deletes train schedule records.
UC-12	Manage Fares & Quota	Admin	Administrator updates class fares and seat quota configuration.
UC-13	Generate Reports	Admin	Administrator generates occupancy or revenue reports.



4.2 Booking State Machine

The application transitions between the following states during a typical booking session:

State	Trigger to Enter	Trigger to Leave
MAIN_MENU	Application launch or session end	Login / Register / Search Trains / Admin Login selected
SEARCH_RESULTS	Search Trains submitted from MAIN_MENU	Train and class selected (→ SEAT_SELECTION)
SEAT_SELECTION	Train/class selected from SEARCH_RESULTS	Seats confirmed (→ PASSENGER_DETAILS) or hold expires (→ SEARCH_RESULTS)
PASSENGER_DETAILS	Seats confirmed in SEAT_SELECTION	Passenger details submitted (→ PAYMENT_PENDING)
PAYMENT_PENDING	Passenger details submitted	Payment succeeds (→ CONFIRMED/WAITLISTED) or fails (→ SEAT_SELECTION)
CONFIRMED / WAITLISTED / RAC	Payment succeeds	Cancellation requested (→ CANCELLED) or departure passes (terminal)
CANCELLED	Cancellation requested on a Confirmed/Waitlisted/RAC booking	Refund processed (→ REFUND_PROCESSED)
REFUND_PROCESSED	Cancellation accepted and refund calculated	Terminal state
ADMIN_MENU	Admin Login authenticated	Manage Trains / Manage Fares / Generate Reports selected, or logout (→ MAIN_MENU)

5. System Features

This section organises the functional requirements by system feature, the major services provided by the product. Each feature includes its priority, a stimulus/response sequence, and the associated functional requirements table.

5.1 User Registration & Authentication

Description and Priority

This feature covers passenger account creation, login, and basic account-lockout protection. Priority: HIGH. It is the entry point that gates access to booking-related features.

Stimulus/Response Sequences

- User launches the application → System displays the Main Menu (Register, Login, Search Trains, Admin Login, Exit).
- User selects Register → System prompts for name, email, phone number and password → validates and creates the account.
- Registered user selects Login → System prompts for email/username and password → authenticates and starts a session.
- User enters an incorrect password 5 consecutive times → System locks the account for 15 minutes and logs the event.

Functional Requirements

Req ID	Requirement	Priority
FR-01	The system shall allow a new passenger to register using name, email, phone number and password.	High
FR-02	The system shall reject registration if the supplied email is already associated with an existing account.	High
FR-03	The system shall authenticate a registered passenger via email/username and password before allowing any booking operation.	High
FR-04	The system shall lock a user account for 15 minutes after 5 consecutive failed login attempts and record an audit event.	Medium

5.2 Train Search

Description and Priority

Allows a passenger to search for trains between two stations on a given date. Priority: HIGH.

Stimulus/Response Sequences

- User selects Search Trains → System prompts for source station, destination station and date of journey.
- System queries the train-schedule data store → System displays matching trains with timings, duration and per-class fares.
- No trains match the query → System displays a "No trains found" message.

Functional Requirements

Req ID	Requirement	Priority
FR-05	The system shall allow a passenger to search trains by source station, destination station and date of journey.	High

Req ID	Requirement	Priority
FR-06	The system shall display, for each matching train, the available classes (Sleeper, AC 3-Tier, AC 2-Tier, Chair Car) with the fare for each class.	High
FR-07	The system shall display a clear "No trains found" message when no train matches the search criteria.	Medium

5.3 Seat Availability & Booking

Description and Priority

Covers checking live seat availability, temporarily holding seats during checkout, and creating a booking record with a unique PNR. Priority: HIGH.

Stimulus/Response Sequences

- User selects a train and class → System displays real-time seat availability for that train/date/class.
- User confirms seat selection → System places a 5-minute soft hold on the requested seat(s).
- User enters passenger details (name, age, gender) for each traveller → System validates the details.
- User confirms the booking → System creates a booking record with a unique PNR and status Confirmed, Waitlisted, or RAC.

Functional Requirements

Req ID	Requirement	Priority
FR-08	The system shall show real-time seat availability for a selected train, date and class before booking is confirmed.	High
FR-09	The system shall place a temporary hold (soft lock) on selected seat(s) for 5 minutes while the passenger completes the booking.	High
FR-10	The system shall allow entry of passenger name, age and gender for up to 6 travellers per booking.	High
FR-11	The system shall generate a unique 10-digit PNR (Passenger Name Record) for every confirmed, waitlisted or RAC booking.	High
FR-12	The system shall place a booking on a waitlist with a waitlist position when no confirmed seat is available.	Medium
FR-13	The system shall automatically upgrade a waitlisted booking to Confirmed status when a previously booked seat is cancelled.	Medium

5.4 Payment Processing

Description and Priority

Calculates the total fare and processes payment through a simulated payment module, confirming the booking on success. Priority: HIGH.

Stimulus/Response Sequences

- Passenger details are confirmed → System calculates the total fare (base fare × number of passengers + applicable surcharge).
- System prompts for payment via the simulated Payment Module → User submits payment details.
- Payment succeeds → System updates booking status to Confirmed/Waitlisted and generates a receipt.
- Payment fails → System releases the seat hold and returns the user to the booking screen.

Functional Requirements

Req ID	Requirement	Priority
FR-14	The system shall calculate the total fare as base fare per class multiplied by the number of passengers, plus any applicable surcharge.	High
FR-15	The system shall process payment through a simulated Payment Module and confirm the booking only on successful payment.	High
FR-16	The system shall generate and persist a payment receipt with a unique transaction ID upon successful payment.	Medium

5.5 Cancellation & Refund

Description and Priority

Allows a passenger to cancel a ticket before departure and receive a refund according to a configurable cancellation-fee policy. Priority: HIGH.

Stimulus/Response Sequences

- User enters a PNR to cancel → System validates ownership of the booking and checks that departure has not passed.
- System calculates the refund amount based on the configured cancellation-fee slab (time remaining before departure).
- System updates the booking status to Cancelled and releases the seat back to inventory, triggering a waitlist upgrade check.

Functional Requirements

Req ID	Requirement	Priority
FR-17	The system shall allow a passenger to cancel a Confirmed, Waitlisted or RAC ticket before the scheduled departure time.	High
FR-18	The system shall calculate the refund amount according to a configurable cancellation-fee slab based on time before departure.	High
FR-19	The system shall release a cancelled seat back to the seat inventory and trigger a waitlist auto-upgrade check.	High

5.6 PNR Status & Booking History

Description and Priority

Lets a passenger check the current status of a booking and review past and upcoming journeys. Priority: MEDIUM.

Stimulus/Response Sequences

- User enters a PNR → System returns the current booking status (Confirmed/Waitlisted/RAC/Cancelled) with seat and coach details.
- User opens Booking History → System displays all bookings for the logged-in passenger, filterable by upcoming/past.

Functional Requirements

Req ID	Requirement	Priority
FR-20	The system shall allow a passenger to check booking status by entering the PNR number.	High

Req ID	Requirement	Priority
FR-21	The system shall display a passenger's complete booking history with filters for upcoming and past journeys.	Low
FR-22	The system shall allow booking history to be sorted by journey date.	Low

5.7 E-Ticket Generation

Description and Priority

Generates a formatted e-ticket once a booking is confirmed, which the passenger can save or print. Priority: MEDIUM.

Stimulus/Response Sequences

- Booking is confirmed → System generates a formatted e-ticket containing PNR, train, seat, fare and passenger details.
- User selects Download/Print Ticket → System writes the e-ticket to a local file for saving or printing.

Functional Requirements

Req ID	Requirement	Priority
FR-23	The system shall generate a formatted e-ticket (PNR, train, seat, fare and passenger details) upon booking confirmation.	Medium
FR-24	The system shall allow the passenger to save or print the e-ticket as a local file.	Medium

5.8 Admin: Train & Fare Management

Description and Priority

Allows an administrator to maintain train schedules, fares and seat quotas. Priority: HIGH.

Stimulus/Response Sequences

- Admin selects Admin Login → System authenticates using separate admin credentials.
- Admin adds or edits a train schedule record (train number, name, source, destination, timing, seat capacity per class).
- Admin updates the fare for a class or the seat quota for a category.

Functional Requirements

Req ID	Requirement	Priority
FR-25	The system shall authenticate an administrator via a separate admin login before allowing schedule or fare changes.	High
FR-26	The system shall allow an administrator to add, edit or delete train schedule records.	High
FR-27	The system shall allow an administrator to set or update the fare for each travel class.	High
FR-28	The system shall allow an administrator to configure the seat quota per category (General, Tatkal, Ladies, RAC).	Medium

5.9 Admin: Reports

Description and Priority

Provides administrators with operational visibility through occupancy and revenue reports. Priority: MEDIUM.

Stimulus/Response Sequences

- Admin selects Generate Report → System prompts for a train/date or date range.
- System computes and displays the requested occupancy or revenue report.

Functional Requirements

Req ID	Requirement	Priority
FR-29	The system shall generate a seat-occupancy report for a specified train and date.	Medium
FR-30	The system shall generate a revenue report for a specified date range.	Medium

6. Other Nonfunctional Requirements

6.1 Performance Requirements

NFR ID	Requirement	Metric
NFR-01	The system shall return train search results within 2 seconds for a typical dataset (≤ 200 trains).	Response time ≤ 2 s
NFR-02	The system shall complete a booking transaction (seat hold to confirmation) within 3 seconds under normal conditions.	Transaction time ≤ 3 s
NFR-03	Application startup (launch to Main Menu) shall complete within 3 seconds on the target hardware.	Start time ≤ 3 s
NFR-04	Booking, seat-inventory and user data file read/write operations shall complete within 200 ms.	File I/O ≤ 200 ms
NFR-05	The executable's memory footprint shall not exceed 128 MB during any active session.	RAM ≤ 128 MB

6.2 Safety Requirements

- The application shall not cause any data loss to existing user, train, or booking files on the host machine.
- If a data file becomes corrupt (e.g., due to a crash during write), the system shall detect this on next launch and re-initialise the affected store safely, without an unhandled exception.
- The system shall not enter an infinite loop or hang under any normal sequence of user inputs.

6.3 Security Requirements

Security Objectives:

- Confidentiality: Protect passenger personal data (name, contact details) and payment information from unauthorised access or disclosure.
- Integrity: Ensure booking, seat-inventory and payment records cannot be tampered with, guaranteeing that confirmed bookings and fare charges remain accurate.

Security Requirements:

Req ID	Requirement	Priority
SEC-01	The system shall store passwords using a salted hash (e.g., SHA-256 with per-user salt) and never store plaintext passwords.	High
SEC-02	All local data files (users.dat, bookings.dat, trains.dat) shall be written only to the application's own directory, with no access to system-level directories.	High
SEC-03	All user input parsed from files or the console shall be validated and length-checked to prevent buffer overflow or malformed-record injection.	High
SEC-04	A logged-in session shall automatically expire after 15 minutes of inactivity, requiring re-authentication.	Medium
SEC-05	Each data file shall include a basic checksum or record-count check to detect accidental corruption on load.	Medium

6.4 Software Quality Attributes

Quality Attribute	Description
Maintainability	The codebase shall be organised into clearly named modules (AuthManager, TrainSearch, BookingManager, SeatInventory, PaymentModule, CancellationManager). Each class shall not exceed 300 lines and all public methods shall have Doxygen-style comments.
Testability	Core business-logic classes (BookingManager, SeatInventory, CancellationManager, PaymentModule) shall be decoupled from any UI layer to allow unit testing without user interaction. Code coverage shall meet ≥70% branch coverage as reported by gcov/lcov.
Reliability	The system shall not crash during any sequence of valid user inputs. Automated tests shall verify booking, cancellation and fare-calculation logic to a pass rate of 100%.
Usability	A new user shall be able to understand the booking flow and complete a reservation within 5 minutes without reading an external manual (in-app instructions provided).
Portability	The codebase shall compile and run on both Windows 10+ and Ubuntu 20.04 LTS using the same CMakeLists.txt.
Extensibility	Adding a new travel class or fare category shall require changes to no more than 2 existing source files (facilitated by an abstract FarePolicy interface).

6.5 Business Rules

- This project is developed exclusively for academic evaluation under the Software Engineering course at PES University. It is not for commercial distribution.
- Each team member must have at least one GitHub commit contributing to at least one functional feature.
- The use of LLM tools (e.g., GitHub Copilot, Claude) is permitted for coding assistance, but every team member must be able to explain all submitted code during evaluation.
- All third-party libraries or assets must be open-source or royalty-free with appropriate attribution in the README.

7. Other Requirements

- The system shall be single-user per running instance; no concurrent multi-machine access is required in this version.

- The application shall include a README.md in the repository root describing: project overview, build instructions (CMake steps), usage/controls, and team member ownership areas.
- The CI/CD pipeline (Jenkins) shall automatically: (a) compile the project, (b) run unit tests, (c) generate code coverage reports, and (d) run SonarQube static analysis on every pull request to the main branch.
- The project must include a Makefile or CMakeLists.txt that builds successfully from a clean checkout without manual dependency installation steps.
- All source files shall include a file-header comment block specifying the author's name, SRN, and a brief description.

Appendix A: Glossary

Term / Acronym	Definition
PNR	Passenger Name Record – the unique 10-digit identifier assigned to every booking.
SRS	Software Requirements Specification – this document.
RAC	Reservation Against Cancellation – a booking status between Waitlisted and Confirmed.
Waitlist	A queue of bookings awaiting a confirmed seat, ordered by booking time.
Quota	A reserved allocation of seats for a specific category (General, Tatkal, Ladies, RAC).
Soft Lock	A temporary hold placed on a seat during checkout to prevent double-booking.
Seat Inventory	The live record of seat occupancy for every train, date and class.
RTM	Requirement Traceability Matrix – a table mapping requirements to design, code, and test artefacts.
SDLC	Software Development Life Cycle – the structured process of planning, designing, implementing, and testing software.
CI/CD	Continuous Integration / Continuous Deployment – automated pipeline for building, testing, and deploying software.
gcov / lcov	GNU coverage tools used to measure code and branch coverage for C++ unit tests.
FR	Functional Requirement – a specific capability the system must provide.
NFR	Non-Functional Requirement – a quality constraint on the system (performance, reliability, etc.).
UC	Use Case – a description of an interaction sequence between an actor and the system.

Appendix B: Field Layouts

B.1 Booking Record Layout (bookings.dat)

The booking file is a plain-text file with one record per line, in the following fixed format:

Format: <PNR>|<TRAIN_NO>|<DATE>|<CLASS>|<STATUS>|<PASSENGER_COUNT>|<TOTAL_FARE>

Field	Max Length	Data Type	Description	Mandatory
PNR	10	String (numeric)	Unique booking identifier	Y
TRAIN_NO	5	String	Train number for the booking	Y

Field	Max Length	Data Type	Description	Mandatory
DATE	10	String (YYYY-MM-DD)	Date of journey	Y
CLASS	3	String (SL/3A/2A/CC)	Booked travel class	Y
STATUS	10	String	Confirmed / Waitlisted / RAC / Cancelled	Y
PASSENGER_COUNT	1	Integer (1-6)	Number of passengers on the booking	Y
TOTAL_FARE	8	Unsigned Integer	Total fare charged for the booking	Y

Example bookings.dat content:

1042837591|12801|2026-11-14|3A|Confirmed|2|1780

1042837602|16536|2026-11-15|SL|Waitlisted|1|450

B.2 System Configuration Parameters

Parameter	Data Type	Default Value	Description
MAX_PASSENGERS_PER_BOOKING	int	6	Maximum travellers allowed per booking
SEAT_HOLD_DURATION_SEC	int	300	Duration of the soft lock on selected seats
PNR_LENGTH	int	10	Number of digits in a generated PNR
SESSION_TIMEOUT_MIN	int	15	Minutes of inactivity before session expiry
CANCELLATION_FEE_TIER1_PCT	int	25	Cancellation fee (%) if cancelled >24h before departure
CANCELLATION_FEE_TIER2_PCT	int	50	Cancellation fee (%) if cancelled within 24h of departure
MAX_LOGIN_ATTEMPTS	int	5	Failed logins allowed before account lockout
ACCOUNT_LOCKOUT_MIN	int	15	Lockout duration after max failed logins

Appendix C: Requirement Traceability Matrix

Req ID	Brief Description	Use Case	Design Module	Code File (Planned)	Test Case ID
FR-01	The system shall allow a new passenger to register using name, em...	UC-01	AuthManager	Auth.cpp	UT-01
FR-02	The system shall reject registration if the supplied email is alr...	UC-01	AuthManager	Auth.cpp	UT-02
FR-03	The system shall authenticate a registered passenger via email/us...	UC-02	AuthManager	Auth.cpp	UT-03
FR-04	The system shall lock a user account for 15 minutes after 5 conse...	UC-02	AuthManager	Auth.cpp	UT-04
FR-05	The system shall allow a passenger to search trains by source sta...	UC-03	TrainSearch	Search.cpp	UT-05
FR-06	The system shall display, for each matching train, the available ...	UC-03	TrainSearch	Search.cpp	UT-06

Req ID	Brief Description	Use Case	Design Module	Code File (Planned)	Test Case ID
FR-07	The system shall display a clear "No trains found" message when n...	UC-03	TrainSearch	Search.cpp	UT-07
FR-08	The system shall show real-time seat availability for a selected ...	UC-04	SeatInventory	SeatInventory.cpp	UT-08
FR-09	The system shall place a temporary hold (soft lock) on selected s...	UC-04	SeatInventory	SeatInventory.cpp	UT-09
FR-10	The system shall allow entry of passenger name, age and gender fo...	UC-05	BookingManager	Booking.cpp	UT-10
FR-11	The system shall generate a unique 10-digit PNR (Passenger Name R...	UC-05	BookingManager	Booking.cpp	UT-11
FR-12	The system shall place a booking on a waitlist with a waitlist po...	UC-05	BookingManager	Booking.cpp	UT-12
FR-13	The system shall automatically upgrade a waitlisted booking to Co...	UC-07	BookingManager	Booking.cpp	UT-13
FR-14	The system shall calculate the total fare as base fare per class ...	UC-06	PaymentModule	Payment.cpp	UT-14
FR-15	The system shall process payment through a simulated Payment Modu...	UC-06	PaymentModule	Payment.cpp	UT-15
FR-16	The system shall generate and persist a payment receipt with a un...	UC-06	PaymentModule	Payment.cpp	UT-16
FR-17	The system shall allow a passenger to cancel a Confirmed, Waitlis...	UC-07	CancellationManager	Cancellation.cpp	UT-17
FR-18	The system shall calculate the refund amount according to a confi...	UC-07	CancellationManager	Cancellation.cpp	UT-18
FR-19	The system shall release a cancelled seat back to the seat invent...	UC-07	CancellationManager	Cancellation.cpp	UT-19
FR-20	The system shall allow a passenger to check booking status by ent...	UC-08	PNRService	PNRService.cpp	UT-20
FR-21	The system shall display a passenger's complete booking history w...	UC-09	HistoryService	History.cpp	UT-21
FR-22	The system shall allow booking history to be sorted by journey date.	UC-09	HistoryService	History.cpp	UT-22
FR-23	The system shall generate a formatted e-ticket (PNR, train, seat,...	UC-10	TicketPrinter	TicketPrinter.cpp	UT-23
FR-24	The system shall allow the passenger to save or print the e-ticke...	UC-10	TicketPrinter	TicketPrinter.cpp	UT-24
FR-25	The system shall authenticate an administrator via a separate adm...	UC-11	AuthManager	Auth.cpp	UT-25
FR-26	The system shall allow an administrator to add, edit or delete tr...	UC-11	AdminTrainManager	AdminTrain.cpp	UT-26
FR-27	The system shall allow an administrator to set or update the fare...	UC-12	AdminFareManager	AdminFare.cpp	UT-27

Req ID	Brief Description	Use Case	Design Module	Code File (Planned)	Test Case ID
FR-28	The system shall allow an administrator to configure the seat quo...	UC-12	AdminFareManager	AdminFare.cpp	UT-28
FR-29	The system shall generate a seat-occupancy report for a specified...	UC-13	ReportGenerator	Report.cpp	UT-29
FR-30	The system shall generate a revenue report for a specified date r...	UC-13	ReportGenerator	Report.cpp	UT-30
SEC-01	The system shall store passwords using a salted hash (e.g., SHA-2...	UC-01	AuthManager	Auth.cpp	UT-31
SEC-02	All local data files (users.dat, bookings.dat, trains.dat) shall ...	UC-05	FileStore	FileStore.cpp	UT-32
SEC-03	All user input parsed from files or the console shall be validate...	UC-05	FileStore	FileStore.cpp	UT-33
SEC-04	A logged-in session shall automatically expire after 15 minutes o...	UC-02	AuthManager	Auth.cpp	UT-34
SEC-05	Each data file shall include a basic checksum or record-count che...	UC-05	FileStore	FileStore.cpp	UT-35